

CSCI 212

Review Problems (Lectures 1–8)

CSCI 212

Part I: Multiple Choice

Hint: If in doubt you can try it in Eclipse and see what happens. But try to understand why it behaves that way.

Circle the best answer.

1.

```
1 int x = 5;  
2 int y = x;  
3 y = 10;  
4 System.out.println(x);
```

- A. 10
- B. 5
- C. 0
- D. Compile error

2.

```
1 int x = 3;  
2 x = x + 2;  
3 System.out.println(x);
```

- A. 3
- B. 5
- C. 2
- D. Compile error

3.

```
1 int[] a = new int[4];  
2 System.out.println(a[2]);
```

- A. null
- B. 0
- C. undefined
- D. Compile error

4.

```
1 String s = "hello";  
2 System.out.println(s.substring(1,4));
```

- A. hel
- B. ell
- C. ello
- D. he

5.

```
1 String s = "CSCI212";  
2 System.out.println(s.indexOf("2"));
```

- A. 2
- B. 4
- C. 6
- D. 8

6. Accessing a[10] when a has length 5 results in:

- A. 0
- B. null
- C. ArrayIndexOutOfBoundsException
- D. Compile error

7. Arrays determine their size using:

- A. size()
- B. length field
- C. getLength()
- D. count()

8. Which gives the number of elements in an ArrayList?

- A. length
- B. size()
- C. getSize()
- D. count()

9.

```
1 int[] a = {1,2,3};  
2 int[] b = a;  
3 b[0] = 10;  
4 System.out.println(a[0]);
```

- A. 1
- B. 10
- C. 0
- D. Compile error

10.

```
1 Counter c1 = new Counter();  
2 Counter c2 = c1;  
3 c2.increment();  
4 System.out.println(c1.getCount());
```

- A. 0
- B. 1
- C. 2
- D. Compile error

11.

```
1 Counter a = new Counter();  
2 Counter b = new Counter();  
3 b = a;
```

How many Counter objects exist?

- A. 1
- B. 2
- C. 3
- D. 0

12. What happened to the original object created for b?

- A. It was copied
- B. It became garbage
- C. It moved to stack
- D. Nothing

13.

```
1 Counter a = new Counter();  
2 Counter b = new Counter();  
3 a = b;  
4 b = new Counter();
```

How many objects exist?

- A. 1
- B. 2
- C. 3
- D. 4

14. In Question 13, how many objects are garbage?

- A. 0
- B. 1
- C. 2
- D. 3

15. An alias is:

- A. A copy of an object
- B. A second reference to the same object
- C. A new object
- D. A static variable

16.

```
1 static void addOne(int x){  
2     x = x + 1;  
3 }  
4 int a = 5;  
5 addOne(a);  
6 System.out.println(a);
```

- A. 6
- B. 5
- C. 0
- D. Compile error

17.

```
1 static void addOne(Counter c){  
2     c.increment();  
3 }  
4 Counter x = new Counter();  
5 addOne(x);  
6 System.out.println(x.getCount());
```

- A. 0
- B. 1
- C. 2
- D. Compile error

18.

```
1 static void change(Counter c){  
2     c = new Counter();  
3     c.increment();  
4     c.increment();  
5 }  
6 Counter x = new Counter();  
7 change(x);  
8 System.out.println(x.getCount());
```

- A. 0
- B. 2
- C. Compile error
- D. Undefined

19. Objects created with **new** are stored:

- A. On the stack
- B. On the heap
- C. In registers
- D. Inside methods

20. An object becomes eligible for garbage collection when:

- A. It goes out of scope
- B. No references point to it
- C. new is called again
- D. It is set to 0

21. Autoboxing is:

- A. Primitive to object automatically
- B. Object to primitive automatically
- C. Sorting automatically
- D. Garbage collection

22.

```
1 ArrayList<String> a =  
2     new ArrayList<String>();  
3 ArrayList<String> b = a;  
4 b.add("Song");  
5 System.out.println(a.size());
```

- A. 0
- B. 1
- C. Compile error
- D. Undefined

Part II: Memory Diagrams

23. Draw the memory diagram:

```
1 Counter a = new Counter();  
2 Counter b = a;  
3 a.increment();  
4 b.increment();
```

24. Draw the memory diagram and indicate garbage:

```
1 Counter a = new Counter();  
2 Counter b = new Counter();  
3 a = b;
```

Part III: Static Function Coding

25. Write a static function `swapFirstTwo` that swaps the first two elements of an `ArrayList<Integer>`.
26. Write a static function `clearPlaylist` that removes all songs from an `ArrayList<String>`.
27. Write a static function `doubleValue` that doubles a primitive `int` parameter.

Part IV: Class Design

28. Write a class `Song` with:

- private `String` `title`
- constructor
- `getTitle()`
- `toString()`

29. Write a class `Playlist` that uses an `ArrayList<Song>` and includes:

- constructor
- `addSong(Song s)`
- `removeSong(int index)`
- `numberOfSongs()`
- `getSongs()`

Part V: Mini Project

30. Mini Project: Playlist Manager

Design three classes:

Song

- `title`
- `toString()`

Playlist

- `ArrayList<Song> songs`
- `addSong()`
- `removeSong()`
- `totalSongs()`
- `printPlaylist()`

Main

- Create at least 3 songs
- Add them to playlist
- Remove one
- Print summary